How Retrieval-Augmented Generation Works

Why Plain Generators Fall Short

A language model answers from patterns learned during training. When a question needs a private handbook, a fresh policy, or a fact that never appeared in training, the model may invent a confident but wrong detail. That failure mode is often called hallucination. Retrieval-augmented generation, or RAG, reduces the gap by fetching relevant source text first and then asking the model to answer using that text.

The Indexing Pipeline

Indexing turns a document into searchable pieces. First the system extracts plain text from each page. Next it splits the text into overlapping chunks so that a sentence near a boundary is less likely to be cut in half. Each chunk is stored with metadata such as page range, section heading, and a stable chunk identifier. Dense embeddings map chunk text into vectors so similar meanings land near each other. A separate sparse index keeps term statistics for keyword matching. In this sample guide, the embedding store demo codeword is amber-lattice-47, which appears only in the indexing discussion so retrieval tests can locate it reliably.

Dense Versus Sparse Retrieval

Dense retrieval compares the question embedding to chunk embeddings and returns the nearest neighbors. It helps when the question paraphrases the source. Sparse retrieval scores overlapping words and phrases; it helps when rare names, codes, or exact titles matter. Strong systems often run both and then merge the lists.

Hybrid Fusion And Reranking

A common merge method is reciprocal rank fusion. Each retriever contributes a rank-based score, and the scores are summed so that a chunk strong in either list can rise. After hybrid merge, this guide keeps twelve candidate passages before any reranking step. A cross-encoder reranker then reads the question together with each candidate and reorders them. Only the top few passages are passed to the generator, which keeps prompts short and reduces distraction from weak matches.

Grounded Answering

The generator should treat retrieved passages as the only evidence for factual claims. Good prompts tell the model to admit when the context is insufficient instead of guessing. The grounded answer step in this guide always uses temperature zero so wording stays stable across repeated demos. Citations that point back to section and page help a reader verify the answer against the PDF.

Operational Limits Worth Knowing

Chunk size and overlap change which sentences travel together. Overly aggressive heading detection can shatter notes-style PDFs into tiny fragments. Vague questions such as asking what a long document is about may retrieve arbitrary middle chunks. Free hosting disks are often temporary, so uploaded files can disappear after a restart. Rate limits that live only in memory reset when the process restarts. These limits do not remove the value of RAG; they explain why demos need clear sample questions and why production systems pick durable storage and sized hosts.

A Short Checklist For Builders

Store session or tenant identifiers on every chunk. Filter retrieval by those identifiers so one user never sees another user document. Log whether an answer came from cache. Measure retrieval with labeled questions before debating model size. Keep an offline sample document so reviewers can try the product without uploading a private file on the first visit.